Comparison of Three Cell Removal Methods in Sudoku Puzzle Generation

Abstract

In a Sudoku generator, creating a uniquely solvable Sudoku begins with filling in an empty 9x9 grid, resulting in a fully solved Sudoku grid. Cells from this grid are then removed to create the final playable puzzle. The difficulty in removal strategies comes from ensuring that the resulting Sudoku puzzle maintains only one solution without drastically increasing generation runtime.

This report compares three Sudoku cell removal methods: Random, Recursive backtracking and Greedy/Iterative. These were used to convert the fully solved Sudoku grid into a playable puzzle, evaluating each method based on the estimated runtime to generate a unique puzzle calculated from the single generation runtime and the probability of generating a unique puzzle.

The results of the investigation demonstrate the trade-offs between ensuring a unique puzzle and generation runtime. Random removal generally performed with the lowest runtime below 40 missing cells, while the Greedy/Iterative approach performed with the lowest runtime within the 40-60 missing cell range where Random and Recursive approaches became computationally expensive. Beyond this point, all three methods became too computationally expensive to generate a unique puzzle.

Table of Contents

1. Introduction

Sudoku is a logic puzzle game in which the player aims to complete a partially solved 9x9 grid by ensuring that each row, column, and box (one of nine 3x3 sub grids) contains each number in the range 1-9 exactly once. A unique Sudoku puzzle is one in which there is only one valid completed grid which satisfies these constraints.

Generating a puzzle, however, has a different challenge than solving one. A Sudoku generator typically starts by generating a completed Sudoku grid, which satisfies the constraints described above and then removes the desired number of cells to create a playable puzzle. As a result, this puzzle is guaranteed to have at least one solution (the original completed grid). However, the puzzle is not necessarily unique as the cell removal may result in other valid grids given the starting cells. There are a variety of different methods to solve the cell removal problem each with their own advantages and disadvantages. The three methods that are investigated in this study are Random removal, Recursive backtracking and Greedy/Iterative. The efficiency of these approaches will be compared by calculating the estimated runtime to generate a valid Sudoku puzzle by measuring the generation runtime and how the probability of generating a unique Sudoku puzzle with each method changes as the number of missing cells increases.

2. Background

2.1 Solution Counter

This method is shared by all three methods to determine whether the given puzzle is unique. The method works by finding the next empty cell in the 9x9 grid. For this cell, it tests each number from 1-9 to find those numbers which are valid for that cell and will attempt to place one of those numbers in the cell. It recursively repeats this step until completing the grid, where it will then return a value of one and then backtracking and emptying the original cell to try another solution.

Importantly, as this method only checks for uniqueness the function will return once it reaches at least two solutions as this is enough to say that the puzzle is no longer unique rather than counting all possible solutions as this becomes computationally expensive as the number of missing cells increases.

2.2 Random Removal

This method works by assigning a number between the range 1-81 to each cell and adding these cell numbers to an array. This array is then shuffled to create a random order of cells and then the first x cells are emptied, where x represents the desired number of missing cells. The solution counter is then used to check for uniqueness and rejects the puzzle if it results in more than one valid grid. This method is the simplest to implement and has the lowest single generation runtime, but this method is likely to require many attempts to produce a unique puzzle, especially as the number of missing cells increases.

2.3 Recursive Backtracking Removal

This method works using a recursive approach. Similarly to Random removal, it assigns a number to each cell and randomly selects a cell from the list of cell numbers and temporarily empties that cell. The solution counter is then used to ensure the current board results in a unique puzzle. If the puzzle is still unique, it will randomly pick a new cell and attempt to empty that cell, whereas if the puzzle is not unique, the algorithm attempts to backtrack, filling back in the empty cell and will try to empty a different cell. These steps are repeated until the desired number of cells are removed. The benefit of this method is that a cell is only permanently removed if the puzzle remains unique. Therefore, any puzzle returned by this method is guaranteed to result in a unique puzzle. However, this approach can result in a rapidly increasing search space as the number of missing cells increases which drastically increases its runtime.

2.4 Greedy/Iterative Method

This method works by iteratively going through the list of shuffled cell numbers and will temporarily empty the current cell and then use the solution counter method to check whether the puzzle is still unique. If the puzzle is still unique, increment the index of the list and try to empty the next number but if the puzzle is not unique fill the number back into the temporary cell and then try to empty the next cell in the list. This is then repeated until the desired number of cells are reached or all cells have been tried. This method only considers each cell at most once and does not backtrack to previous cells. This avoids the rapid recursive explosion but

may fail to remove the desired number of missing cells and in this case will return null to indicate failure. As the method checks for uniqueness after each removal, if it returns a puzzle, it is guaranteed to be unique.

3. Methodology

To test each cell removal method an experiment was conducted using a range of missing cells. A Sudoku puzzle needs at least 17 filled in cells to have a unique solution (McGuire, Tugemann & Civario, 2013) meaning the max range of missing cells is within the range 0-64, as 81 – 64 = 17. Both Random and Greedy/Iterative were tested on the full 0-64 missing cell range whereas the Recursive backtracking method was tested on a smaller range between 0-55 missing cells. This is because beyond this point the explosive nature of the algorithm due to large search space made the runtime too large. Each missing cell was tested 1000 times for each method, and the total generation time was measured. For the Random and Greedy/Iterative method, the program also recorded whether the resulting puzzle was unique.

3.1 Experimental Setup

Component	**Specification**
CPU	Ryzen 7 3700X
RAM	32 GB DDR4
Operating System	Windows 11
Programming Language	Java 26 (Oracle OpenJDK 26.0.2)
IDE	Intellij IDEA

All experiments were conducted on the same computer within the Intellij IDEA to minimise variance in the measured runtimes caused by differences in hardware and software. All methods were tested using the same initial board generator and cell validator to keep these runtimes consistent.

3.2 Experiment Variables

Variable	Type	Description
Number of Missing Cells	Independent	0-64 for Random and Greedy/Iterative. 0-55 for Recursive. Each number tested 1000 times each.
Removal Methods	Independent	Random, Recursive and Greedy/Iterative.
Total trial Runtime	Dependent	Total time for one trial of the experiment, including initial board generation, cell removal and solution counting time.
Uniqueness	Dependent	For Random and Greedy/Iterative. Whether the resulting puzzle only has one valid solution.
Number of trials	Controlled	1000 trials per configuration (for each number of missing cells and for each method).
Sudoku Grid size	Controlled	Standard 9x9 grid.
Solution counter	Controlled	Same solution counter method used for each method
Sudoku Grid generator	Controlled	Same generator used for each method. Returns the initial completed grid.

3.3 Experimental Procedure

Each trial began by generating a complete, valid Sudoku grid. The selected removal method was then run for the given number of missing cells. For Random and Greedy/Iterative, each trial records whether the resulting puzzle is unique whereas the Recursive backtracking method inherently rejects non-unique puzzles. The total trial runtime is measured. This is repeated 1000 times before moving to the next number of missing cells and then repeated for each removal method.

3.4 Uniqueness Measurement and Unique Generation Runtime Calculation

For each trial in Random and Greedy/Iterative removals the uniqueness of the resulting puzzles is recorded. This was then used to then calculate the probability of generating a unique puzzle for a given number of missing cells according to the equation:

$$P(\text{unique}) = \frac{\text{unique puzzles}}{\text{total puzzles}}$$

For example, if at 40 missing cells, Random removal produced 400 unique puzzles out of the 1000 trials, the probability that the generated puzzle is unique with Random removal at 40 missing cells is 0.4.

For each removal and number of missing cells, the probability of uniqueness was calculated along with the mean and median total trial runtime. From this the estimated unique puzzle was calculated according to the equation:

$$R_{\text{unique}} = \frac{R_{\text{single}}}{P(\text{unique})}$$

Where R_{unique} is the estimated runtime to generate a unique puzzle, R_{single} is the mean total trial runtime for that removal method for that given number of missing cells and $P(\text{unique})$ is the probability to generate a unique puzzle for that given number of missing cells.

For example, if at 40 trials Random removal has a probability of generating a unique puzzle of 0.4 and a mean runtime of 14ms, the estimated runtime to generate a unique puzzle = 35ms.

$$\frac{14ms}{0.4} = 35ms$$

Note, the Recursive backtracking method only returns a unique puzzle so its $R_{\text{single}} = R_{\text{unique}}$ as its $P(\text{unique})$ is 1.

3.5 Data processing

Graphs were produced showing the mean and median total trial runtimes against the number of cells. For the Random and Greedy/Iterative methods a graph was produced showing the probability of producing a unique puzzle against number of missing cells. Tables were also drawn up to allow for direct numerical comparison.

To compare all three methods, a final graph and table comparing estimated runtime required to generate a unique puzzle against the number of missing cells were plotted for all three methods.

All source code, raw data and processed tables and graphs used can be found at: https://github.com/tobiayodele/Sudoku/tree/experiments.

4. Results

4.1 Random

The relationship between the number of missing cells and total trial runtime for Random Removal is shown in Figure 1, with the blue line representing the mean total trial runtime and the orange line representing the median total trial runtime. Generally, both lines increase with the mean total trial runtime increasing at a faster rate as the number of missing cells increases, whereas the median total trial runtime remains more stable. The median total trial runtime also begins to generally decrease from around 57 missing cells. At 64 cells the mean total trial runtime was approximately 2072ms which was drastically different from the median total trial runtime of approximately 0.14ms at that number of missing cells.

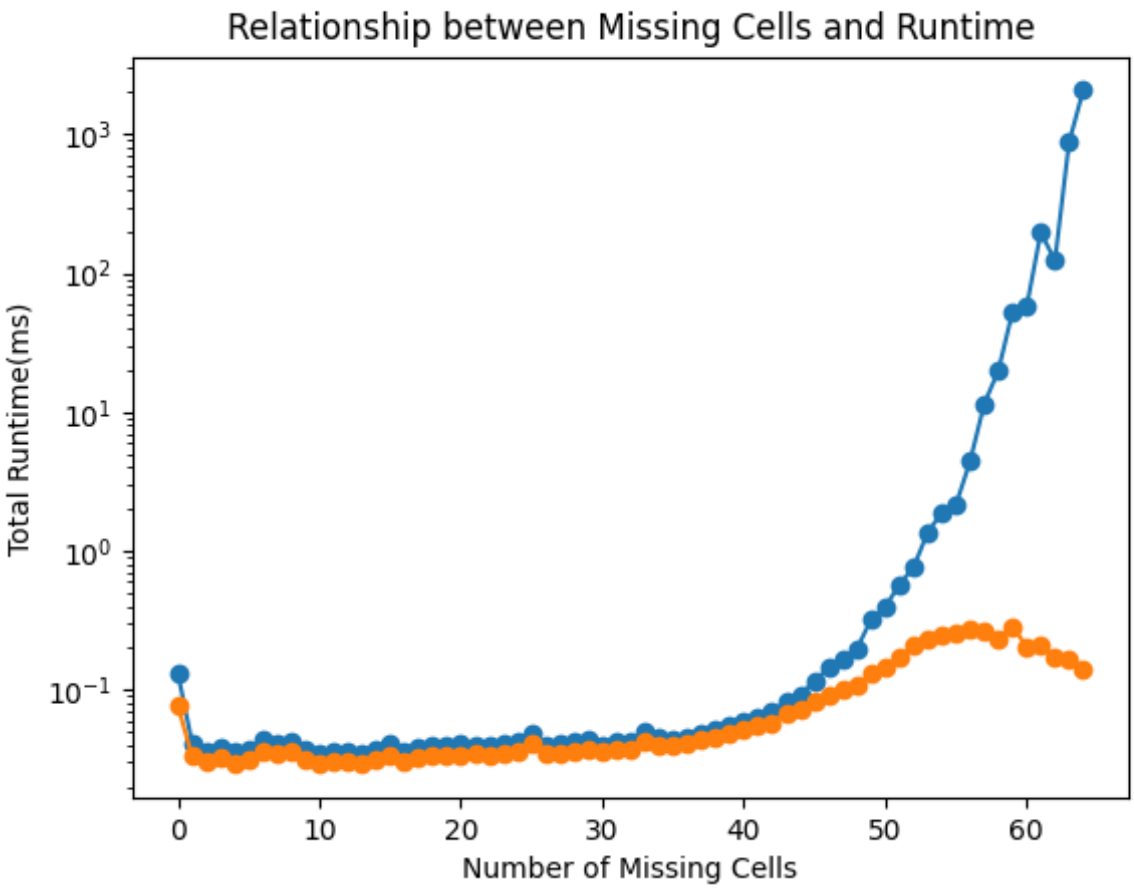

Figure 1: Relationship Between Number of Missing Cells and Runtime for Random removal.

Figure 2 depicts the relationship between the number of missing cells and the probability of finding a unique solution. The graph shows a steep decline starting at around 20 missing cells to a 0% chance at 54 cells.

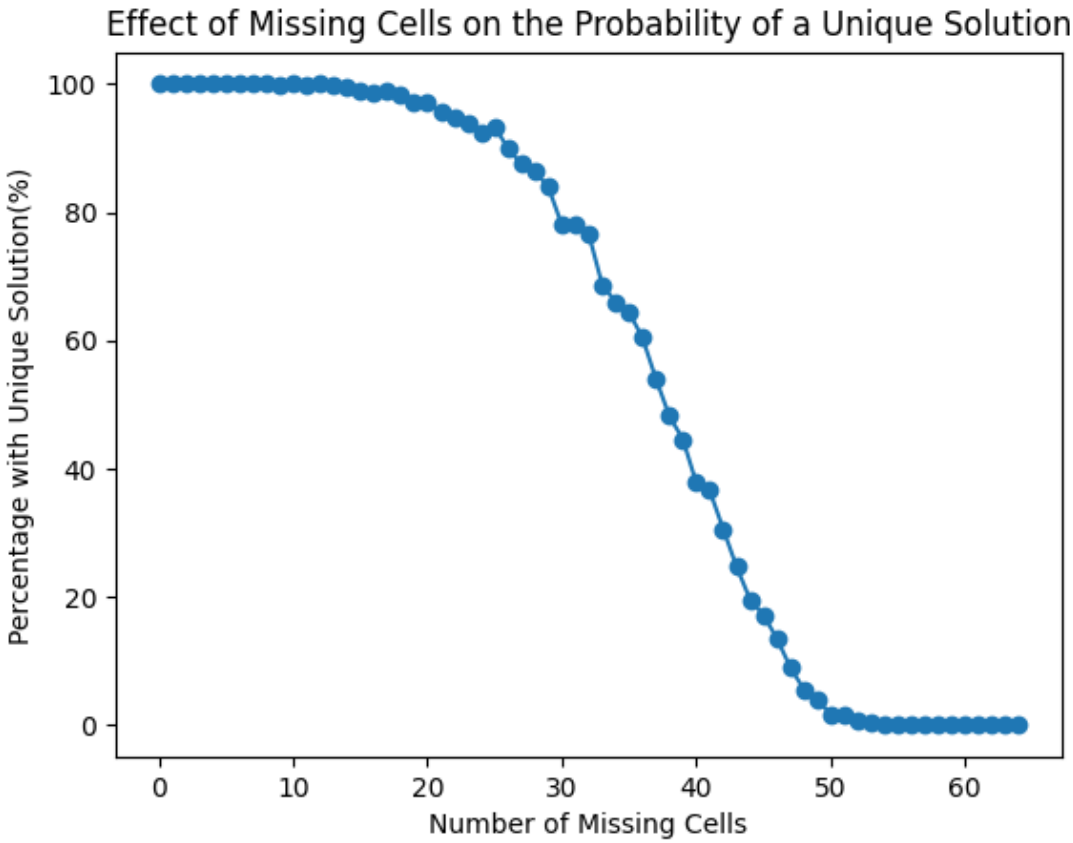

Figure 2: Effect of Missing cells on the Probability of a Unique Solution

Random Removal Table of Results

Missing Cells	Percentage for Unique Solution	Median Runtime(ms)	Mean Runtime(ms)
0	100.0	0.077	0.132
1	100.0	0.033	0.042
2	100.0	0.031	0.037
3	100.0	0.032	0.039
4	100.0	0.029	0.036
5	100.0	0.032	0.038
6	100.0	0.036	0.043
7	100.0	0.035	0.042
8	100.0	0.036	0.043
9	99.9	0.032	0.037
10	100.0	0.03	0.034
11	99.7	0.031	0.036
12	100.0	0.031	0.036
13	99.8	0.03	0.034
14	99.5	0.032	0.037
15	98.9	0.034	0.041
16	98.6	0.031	0.036
17	98.8	0.033	0.038
18	98.2	0.033	0.039
19	97.0	0.033	0.039
20	97.2	0.034	0.041
21	95.7	0.034	0.04
22	94.7	0.034	0.04
23	94.0	0.035	0.042
24	92.5	0.036	0.043
25	93.4	0.04	0.048
26	89.9	0.035	0.04
27	87.5	0.034	0.041
28	86.4	0.036	0.042
29	83.9	0.038	0.043
30	78.0	0.036	0.04
31	78.1	0.037	0.043
32	76.6	0.038	0.043
33	68.7	0.043	0.05
34	66.0	0.04	0.045
35	64.4	0.04	0.044
36	60.4	0.042	0.046
37	54.0	0.043	0.049
38	48.2	0.045	0.051
39	44.6	0.048	0.055
40	37.8	0.051	0.059
41	36.6	0.055	0.063
42	30.6	0.058	0.07
43	24.7	0.068	0.083
44	19.6	0.072	0.093
45	17.2	0.083	0.116
46	13.5	0.092	0.144
47	9.0	0.101	0.167
48	5.6	0.109	0.198
49	4.0	0.132	0.32
50	1.7	0.148	0.393
51	1.5	0.172	0.573
52	0.6	0.211	0.759
53	0.3	0.23	1.359
54	0.0	0.249	1.88
55	0.0	0.253	2.149
56	0.0	0.275	4.407
57	0.0	0.261	11.449
58	0.0	0.235	19.857
59	0.0	0.287	52.671
60	0.0	0.204	57.12
61	0.0	0.21	197.049
62	0.0	0.172	122.721
63	0.0	0.166	874.669
64	0.0	0.14	2071.523

4.2 Recursive

The relationship between the number of missing cells for Recursive backtracking removal and generation is depicted in Figure 3 where the blue line indicates the mean total trial runtime and the orange line indicates the median total trial runtime. Both the mean and median total trial runtime generally increases as the number of cells increases. Both lines begin to increase at a faster rate around the 40 missing cells mark. The largest number of missing cells recorded was 55 with an average total trial runtime of approximately 2418ms which was drastically different to the median total trial runtime of approximately 17ms at 55 missing cells. Beyond this point, the computational cost of the algorithm became too impractical for the experiment.

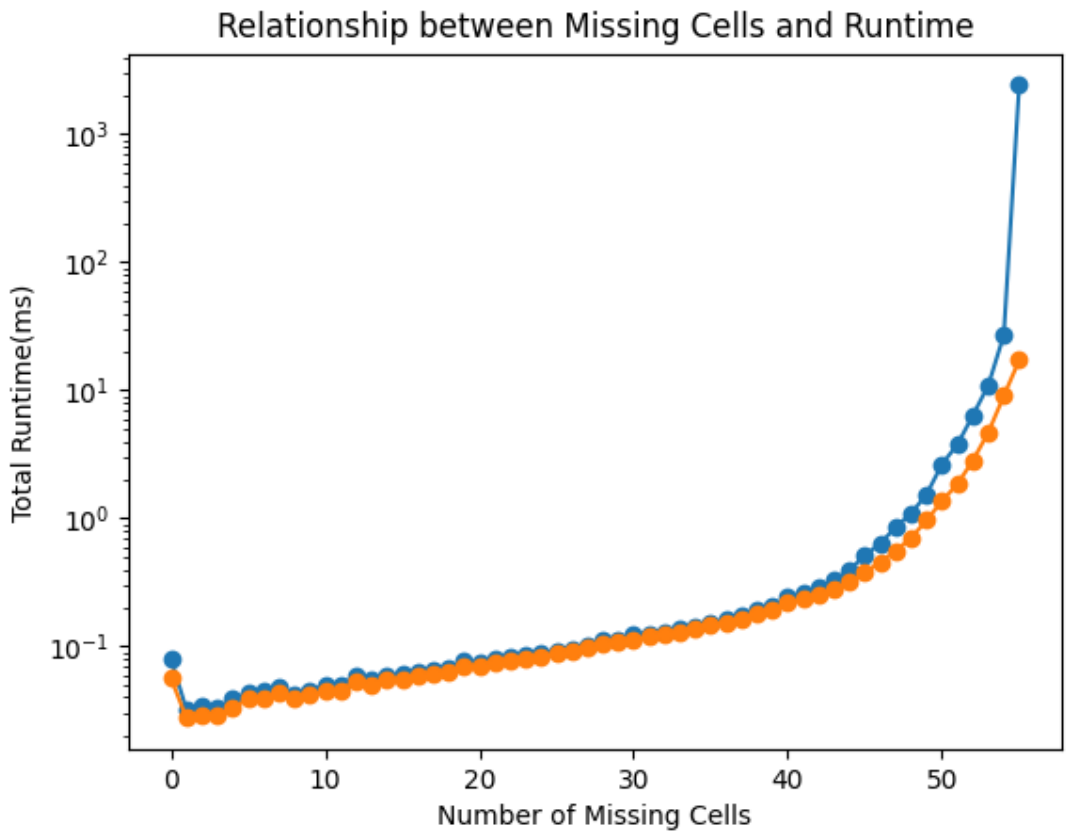

Figure 3: Relationship Between Number of Missing Cells and Runtime for Recursive removal.

Recursive Removal Table of Results

Missing Cells	Median Runtime(ms)	Mean Runtime(ms)
0	0.057	0.08
1	0.028	0.032
2	0.029	0.034
3	0.029	0.033
4	0.033	0.039
5	0.039	0.043
6	0.04	0.045
7	0.043	0.048
8	0.04	0.043
9	0.042	0.045
10	0.045	0.05
11	0.045	0.049
12	0.053	0.059
13	0.05	0.054
14	0.055	0.059
15	0.056	0.061
16	0.059	0.063
17	0.061	0.066
18	0.064	0.067
19	0.07	0.076
20	0.07	0.074
21	0.075	0.08
22	0.078	0.082
23	0.081	0.086
24	0.084	0.088
25	0.088	0.092
26	0.092	0.096
27	0.098	0.102
28	0.104	0.111
29	0.108	0.113
30	0.114	0.123
31	0.118	0.123
32	0.124	0.13
33	0.13	0.137
34	0.137	0.143
35	0.146	0.154
36	0.154	0.163
37	0.165	0.176
38	0.179	0.193
39	0.192	0.209
40	0.219	0.243
41	0.233	0.265
42	0.257	0.288
43	0.28	0.329
44	0.318	0.397
45	0.377	0.513
46	0.456	0.638
47	0.557	0.854
48	0.706	1.097
49	0.966	1.542
50	1.37	2.657
51	1.856	3.779
52	2.829	6.386
53	4.693	10.961
54	9.024	26.929
55	17.178	2417.909

4.3 Iterative/Greedy

Figure 4 shows the relationship between total trial runtime and the number of missing cells. Generally, the total trial runtime increases as the number of missing cells increased, with the mean total trial runtime (represented the blue line) and the median total trial runtime (represented by the orange line) forming the same shape. Both lines begin to increase at a sharper rate at around 45 missing cells. The discrepancy between the mean and the median values increases as the number of missing cells increases and is at its largest within the late 50s until the maximum difference at 64 missing cells. The largest mean total trial runtime was at 60 missing cells where it was approximately 189ms whereas the median total trial runtime was approximately 54ms at this number of missing cells.

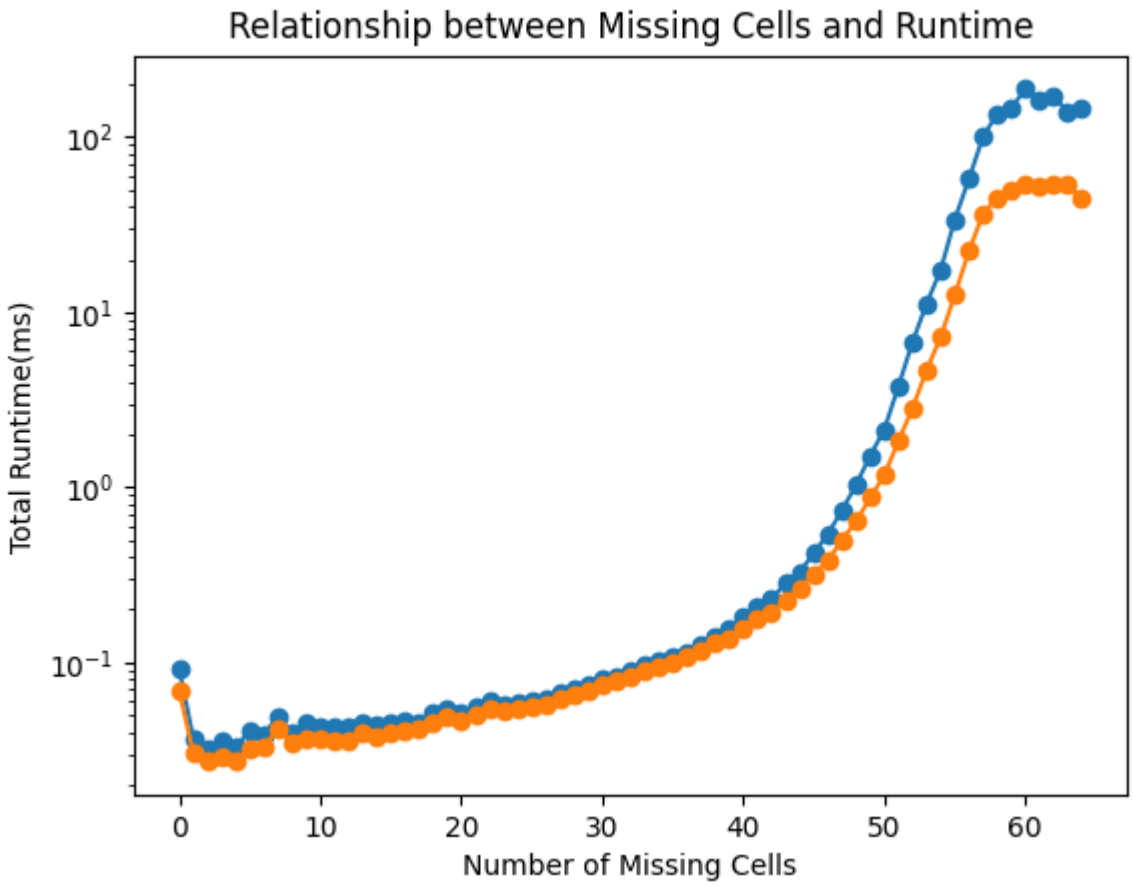

Figure 4: Relationship Between Number of Missing Cells and Runtime for Iterative/Greedy removal.

Figure 5 shows the relationship between the percentage chance of generating a unique sudoku puzzle against the number of missing cells for Iterative/Greedy removal. 100% of puzzles generated were unique until 54 missing cells before rapidly falling to a 0% chance at 61 missing cells.

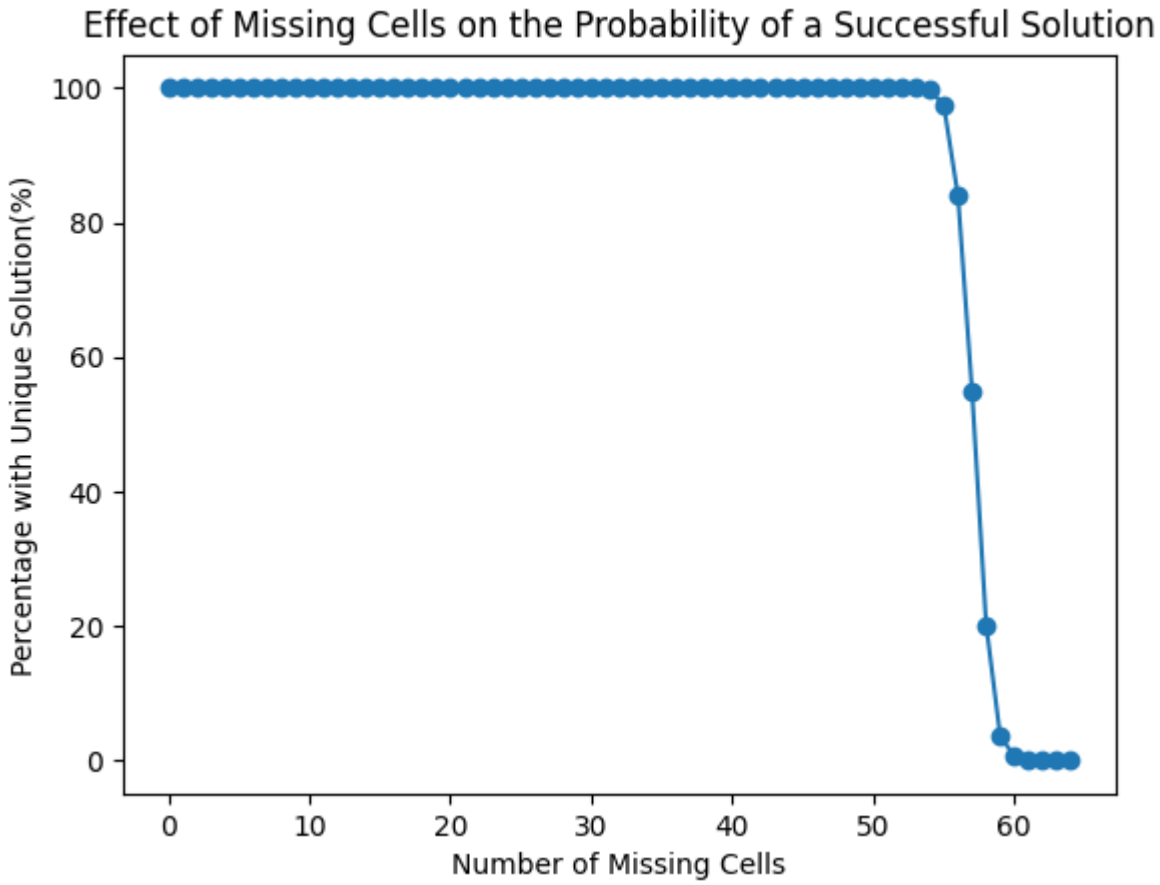

Figure 5: Effect of Missing cells on the Probability of a Unique Solution

Iterative/Greedy Removal Table of Results

Missing Cells	Percentage for Success	Median Runtime(ms)	Mean Runtime(ms)
0	100.0	0.069	0.091
1	100.0	0.03	0.037
2	100.0	0.027	0.032
3	100.0	0.029	0.035
4	100.0	0.027	0.033
5	100.0	0.032	0.041
6	100.0	0.033	0.039
7	100.0	0.042	0.049
8	100.0	0.034	0.039
9	100.0	0.037	0.045
10	100.0	0.037	0.043
11	100.0	0.036	0.042
12	100.0	0.036	0.043
13	100.0	0.039	0.046
14	100.0	0.037	0.044
15	100.0	0.039	0.045
16	100.0	0.04	0.046
17	100.0	0.041	0.046
18	100.0	0.046	0.051
19	100.0	0.049	0.054
20	100.0	0.046	0.052
21	100.0	0.05	0.056
22	100.0	0.054	0.06
23	100.0	0.053	0.057
24	100.0	0.055	0.059
25	100.0	0.056	0.06
26	100.0	0.058	0.062
27	100.0	0.062	0.067
28	100.0	0.066	0.071
29	100.0	0.07	0.075
30	100.0	0.074	0.08
31	100.0	0.078	0.082
32	100.0	0.083	0.089
33	100.0	0.09	0.096
34	100.0	0.094	0.102
35	100.0	0.1	0.107
36	100.0	0.107	0.115
37	100.0	0.117	0.127
38	100.0	0.131	0.142
39	100.0	0.138	0.154
40	100.0	0.158	0.181
41	100.0	0.178	0.209
42	100.0	0.193	0.229
43	100.0	0.226	0.285
44	100.0	0.262	0.329
45	100.0	0.32	0.42
46	100.0	0.382	0.541
47	100.0	0.501	0.739
48	100.0	0.638	1.036
49	100.0	0.874	1.516
50	100.0	1.171	2.138
51	100.0	1.838	3.775
52	100.0	2.828	6.761
53	100.0	4.68	11.236
54	99.8	7.229	17.452
55	97.4	12.598	33.958
56	84.0	22.345	57.729
57	54.9	36.539	100.68
58	20.2	45.254	135.821
59	3.6	49.921	146.439
60	0.7	54.201	188.617
61	0.0	53.037	161.458
62	0.0	53.416	173.59
63	0.0	53.403	138.539
64	0.0	44.284	145.51

Methods Comparison

Figure 6 depicts the relationship between number of missing cells and R_{unique} (the estimate runtime required to generate a unique puzzle) for each removal method. The Random method reached a maximum of 53 missing cells. The Recursive method reached 55 missing cells, and the Iterative method reached 60 missing cells with an estimated runtime of approximately 27000ms.

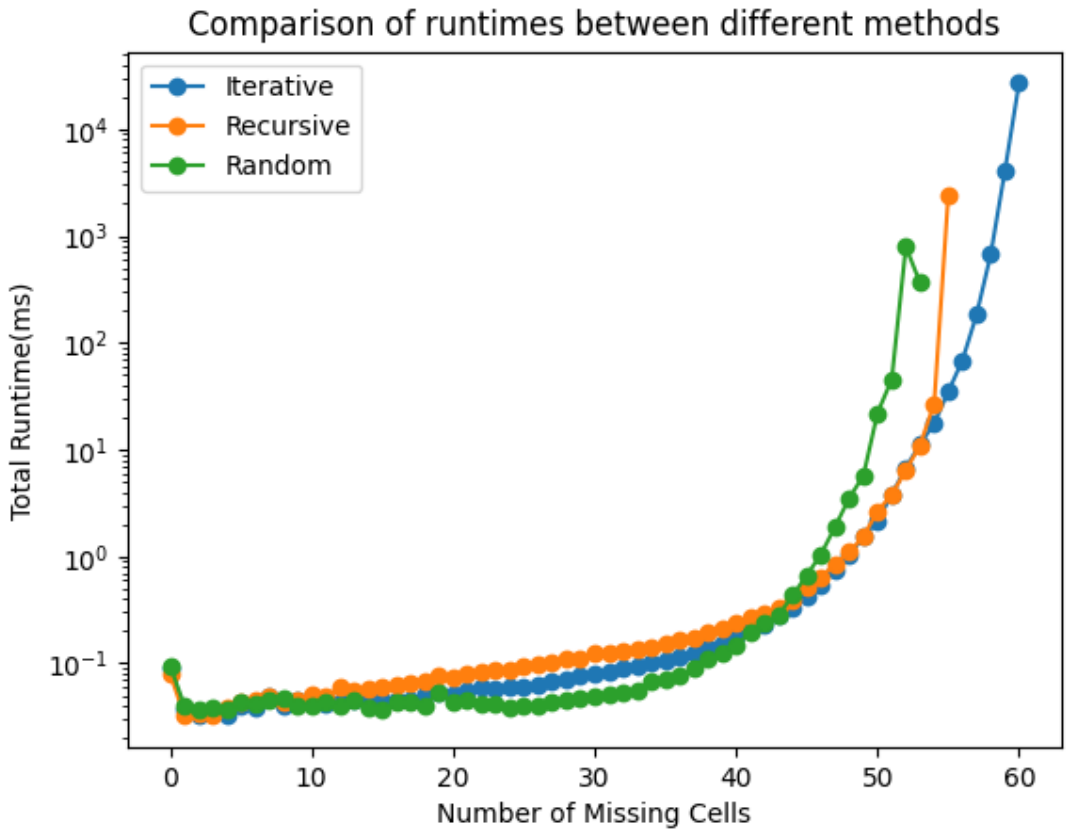

Figure 6: Comparison of runtimes between different methods.

Table Comparison of Methods

The lowest runtime at each number of missing cells is highlighted in green, while the 40-60 missing cell range is highlighted in red.

Missing Cells	Iterative Runtime(ms)	Recursive Runtime(ms)	Random Runtime(ms)
0	0.09	**0.08**	0.096
1	0.04	**0.032**	0.041
2	**0.03**	0.034	0.036
3	0.04	**0.033**	0.039
4	**0.03**	0.039	0.037
5	**0.04**	0.043	0.043
6	**0.04**	0.045	0.041
7	0.05	0.048	**0.046**
8	**0.04**	0.043	0.047
9	**0.04**	0.045	0.04
10	**0.04**	0.05	0.04
11	**0.04**	0.049	0.044
12	**0.04**	0.059	0.04
13	0.05	0.054	**0.045**
14	0.04	0.059	**0.039**
15	0.05	0.061	**0.037**
16	0.05	0.063	**0.043**
17	0.05	0.066	**0.044**
18	0.05	0.067	**0.04**
19	**0.05**	0.076	0.054
20	0.05	0.074	**0.043**
21	0.06	0.08	**0.046**
22	0.06	0.082	**0.043**
23	0.06	0.086	**0.042**
24	0.06	0.088	**0.039**
25	0.06	0.092	**0.041**
26	0.06	0.096	**0.041**
27	0.07	0.102	**0.044**
28	0.07	0.111	**0.046**
29	0.08	0.113	**0.047**
30	0.08	0.123	**0.049**
31	0.08	0.123	**0.051**
32	0.09	0.13	**0.054**
33	0.1	0.137	**0.056**
34	0.1	0.143	**0.067**
35	0.11	0.154	**0.072**
36	0.12	0.163	**0.076**
37	0.13	0.176	**0.09**
38	0.14	0.193	**0.112**
39	0.15	0.209	**0.127**
40	0.18	0.243	**0.148**
41	0.21	0.265	**0.192**
42	**0.23**	0.288	0.242
43	0.29	0.329	**0.28**
44	**0.33**	0.397	0.433
45	**0.42**	0.513	0.667
46	**0.54**	0.638	1.047
47	**0.74**	0.854	1.897
48	**1.04**	1.097	3.526
49	**1.52**	1.542	5.706
50	**2.14**	2.657	22.058
51	3.78	**3.779**	45.288
52	6.76	**6.386**	792.378
53	11.24	**10.961**	365.621
54	**17.49**	26.929	inf
55	**34.86**	2417.909	inf
56	**68.72**	inf	inf
57	**183.39**	inf	inf
58	**672.38**	inf	inf
59	**4067.75**	inf	inf
60	**26945.3**	inf	inf
61	inf	inf	inf
62	inf	inf	inf
63	inf	inf	inf
64	inf	inf	inf

5. Analysis

All three cell removal methods see an initial spike at 0 missing cells (Iterative 0.09ms, Recursive 0.08ms and Random 0.096ms). This is likely attributable to the JVM warm-up and measurement overhead rather than being a representation of the behaviour of the cell removal methods themselves as no cells are removed in these trials.

From 1 to 12 missing cells all three perform almost identically between 0.03ms and 0.06ms runtimes. This tracks as at this number of minimum cells, the probability to generate a non-unique Sudoku puzzle at these missing cells is effectively zero resulting in no meaningful difference between the amount of work each method must do.

Between 13 to 39 missing cells, the Random cell removal performs the fastest. The percentage chance of generating a unique solution drops sharply within this range, but each individual removal attempt is computationally cheap enough that generating multiple puzzles and checking for uniqueness is still faster than both the Greedy/Iterative and Recursive methods. This is most notable at 30 missing cells where Random had a runtime of 0.049ms whereas Iterative was 63% slower at 0.08ms and Recursive was 151% slower at 0.123ms.

The 40-60 missing cell range is the most significant as it is within this range that the algorithms begin to diverge the most. The runtimes of all three methods increases rapidly within this range with the Iterative method having the lowest runtime between 44 and 60 cells, except for 51-53 cells where Recursive is at most 0.374ms faster. Random removal becomes impractical at this range, with an estimated cost of 792.378ms at 52 missing cells and 365.621ms at 53 missing cells before the percentage chance for generating a unique Sudoku puzzle fell to 0% at 54 missing cells. Beyond this the Iterative method begins to outperform the other two methods, being 98.6% faster than Recursive at 55 cells at 34.86ms compared to 2417.909ms. This is because the computational cost of the recursive backtracking method rapidly increases as the search space becomes too high. Beyond this point the recursive backtracking method could not complete in time, leaving the Iterative method as the only one able to complete between 56 and 60 cells.

Beyond 60 missing cells, between 61 and 64 missing cells, the Iterative method was unable to remove the desired number of missing cells after only one pass. This is because there was few or no cells that could be removed without producing a non-unique Sudoku puzzle. This indicates a limitation of the algorithm. As a result, all three methods were unable to produce a unique Sudoku in this range.

Overall, the results indicate that there is not a single most optimal method across all possible number of missing cells. Random removal provides the lowest runtime at low number of missing cells whereas the Iterative method becomes more effective as the number of missing cells increases. The Recursive removal method was strained by the rapidly increasing search space as the number of missing cells increases, leading to substantial increases to runtime. None of the methods were able to produce any unique Sudoku puzzles within the high 61-64 missing cells range indicating a limitation of all three methods.

6. Discussion

Overall, the Greedy/Iterative method performed the fastest across higher number of missing cells. It was able to reach 60 missing cells, whereas the Recursive method only reached 55 missing cells and the Random method reached 53 missing cells.

Random removal was the best-performing method at low number of missing cells because the individual removal attempt runtime is significantly lower than the other two methods as it does not check for uniqueness until all cells are removed. This is significantly faster than checking at each removal. However, as cells are randomly removed, there is an increasing probability of leading to a non-unique solution meaning that multiple generation attempts must be made, leading to both Greedy/Iterative and Recursive becoming more competitive.

Iterative removal becomes more suitable as the number of missing cells increases as unlike Random removal the algorithm continually runs the solution checker at each temporary removal so that it does not need to generate multiple puzzle attempts before producing a unique puzzle. In addition to this, rather than recursive backtracking, the method performs only one pass through the cells to avoid the struggle of a rapidly expanding search space which constrained the Recursive method at higher number of missing cells.

The main limitation of the Greedy/Iterative method is that it only completed 1 pass through the Sudoku cells. This means that as the number of cells increases, particularly above 60 missing cells, the algorithm is likely to be unable to fill the desired number of missing cells. The efficiency of the method may be improved by doing multiple passes so that cells that were originally rejected can be tried again in case they still maintain a unique solution.

Furthermore, the solution counter may be improved by implementing a choosing heuristic rather than always trying the next empty cell. An example is ordering the free empty cells by the amount of valid numbers which can go into that cell and prioritising the cells with the lowest amount of values. This means that cells with the largest constraints are considered first which attempts to identify invalid branches in the search tree first which is likely to improve the runtimes of all three methods.

Each cell removal method was tested for 1000 trials at each number of missing cells to provide a substantial sample size for comparing performance. Further trials could be conducted to reduce the effect of unusually high or low trials. Furthermore, to reduce the effect of JVM warm up overhead, initial executions could be performed before measurements are recorded. The experiment could also be performed on different hardware to see if the same trends are seen.

7. Conclusion

This report compared Random, Recursive and Greedy/Iterative cell removal methods in unique Sudoku puzzle generation to identify the most efficient method as the number of missing cells increases. No single method performed the best across all missing cells, with Random cell removal performing best at lower numbers of missing cells and Greedy/Iterative performing better as the number of missing cells increased. The Recursive backtracking method was constrained by a rapidly growing search space resulting in significantly higher runtimes as the number of missing cells increased. However, none of the methods were able to generate a sudoku with more than 60 missing cells.

These results suggest that a Sudoku generator which has difficulty levels within 40 to 60 missing cells should use the Greedy/Iterative removal method as it performed with the lowest runtime within this range. Future work may include a custom choosing heuristic for the solution checker to provide improvements for all three methods.

8. References

1. McGuire, G., Tugemann, B. and Civario, G. (2013). There is no 16-Clue Sudoku: Solving the Sudoku Minimum Number of Clues Problem. *arXiv:1201.0749 [cs]*. [online] Available at: https://arxiv.org/abs/1201.0749 [Accessed 2 Sept. 2026].